

Abstract

This is the report for Exercise 2. In this report we showcase the results of a hybrid MPI/OpenMP implementation of Exercise 2c, as described in the [GitHub repository](#).

1 Mandelbrot Set

1.1 Algorithm

The Mandelbrot set algorithm maps each pixel (x, y) to its corresponding point c in the complex plane. For each point, it iteratively computes $z_{n+1} = z_n^2 + c$ until either $|z_n| \geq 2$ or the maximum number of iterations is reached. For each point, the iteration count at which the sequence escapes is stored in the corresponding pixel. If the maximum number of iterations is reached without escape, the pixel is assigned the value 0.

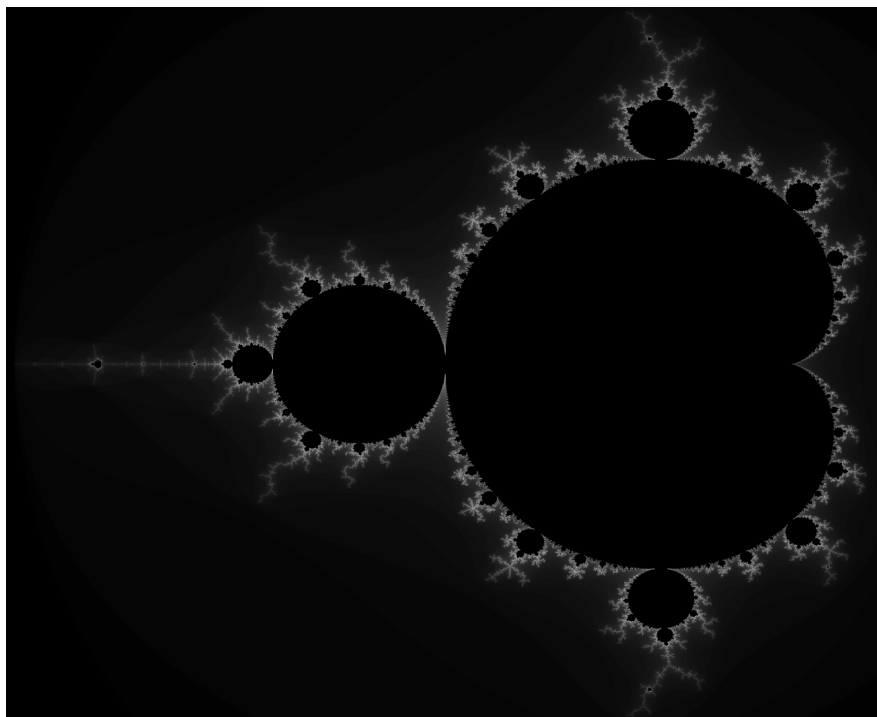


Figure 1: The Mandelbrot fractal.

1.2 Initial considerations

The Mandelbrot set computation is trivially easy to parallelize: every pixel computed is assumed to be completely independent of the others. That makes the action of spreading the workload extremely easy among threads and processes alike.

However, there is an important consideration to make: **the Mandelbrot set's complexity is not evenly distributed** among the pixels, which means the implementation has to take care of a highly irregular computational workload. Naively partitioning the complex plane and assigning a fixed number of pixels to each thread or process can lead to significant load imbalance.

2 Benchmarking

Before discussing the OpenMP/MPI implementations, it's important to define what kind of benchmark they will be tested against. The weak and strong scaling of both OpenMP and MPI are tested on a computationally demanding region of the Mandelbrot set:

$$z_{bl} = 0.35787121 + 0.1081397025i, z_{tr} = z_{bl} + (1 + i) * 10^{-8}, I_{max} = 3000 \quad (1)$$

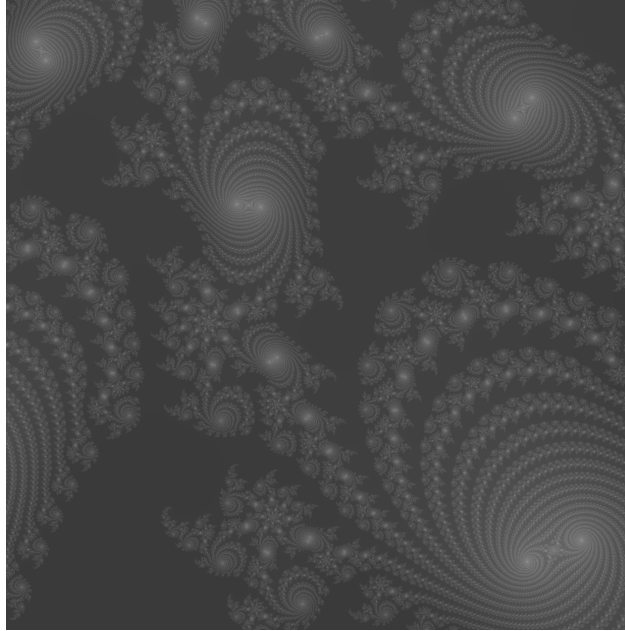


Figure 2: Mandelbrot section used for benchmarking.

Each configuration was executed 5 times across 4 THIN nodes of the ORFEO cluster, and the measured wall-clock execution times were averaged to reduce the impact of run-to-run variability. The benchmark has been run on the /fast partition for faster I/O operations.

2.1 Scaling analysis

The benchmark tests for two kinds of scaling: **strong** and **weak** scalability.

- **Strong scalability** describes how the execution time of a fixed-size problem changes as the number of computational resources is increased. In this specific case, the test consists in a **2048×2048 region** of the computationally demanding Mandelbrot area.
- **Weak scalability** describes how the execution time changes as both the problem size and the number of computational resources are increased proportionally, such that the amount of work assigned to each resource remains approximately constant. In this case, the baseline problem size is a **512×512 region**. The side lengths of the region are **increased proportionally to the square root of the number of processes**, ensuring that the total number of pixels, and therefore the computational workload, grows approximately linearly with the number of processes. Ideally, the execution time should remain approximately constant as the number of processes increases.

3 OpenMP

3.1 Implementation

OpenMP is the most straightforward parallelization to implement, since OpenMP already provides dynamic scheduling through the **schedule(dynamic)** clause and it can be applied directly to the y, x loop over the pixels. Although dynamic scheduling introduces additional scheduling overhead, the improved load balancing

significantly outweighs this cost for this workload.

To parallelize the code we wrap the y, x for loops with an OMP PARALLEL FOR directive with the following parameters:

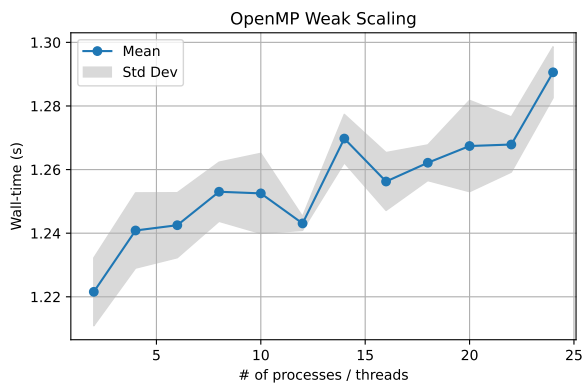
```
#ifndef _OPENMP
#pragma omp parallel for \
    collapse(2) \
    schedule(dynamic, 64) \
    default(none) \
    shared(pixels, btm_left, top_right, dy, dx, i_max, n_x, n_y, tile) \
    proc_bind(spread)
#endif
```

Figure 3: OpenMP directive.

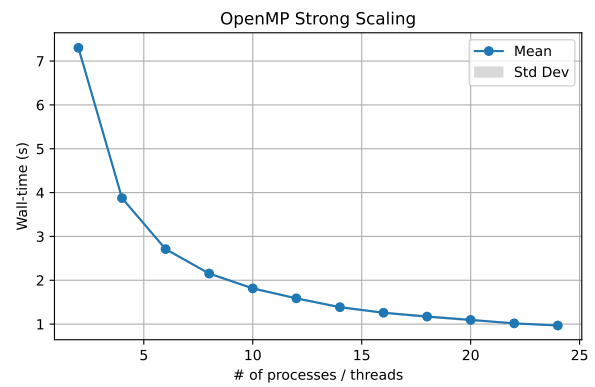
Several chunk sizes were evaluated experimentally. A chunk size of 64 provided the lowest average execution time among the tested configurations, representing a good trade-off between load balancing and scheduling overhead. Smaller chunks improve load balancing but increase scheduling overhead, while larger chunks reduce scheduling overhead at the cost of potentially greater load imbalance. The collapse(2) clause combines the two nested loops into a single iteration space, allowing OpenMP to distribute individual pixel computations more flexibly among threads.

3.2 Scaling

Both weak and strong OpenMP scaling show good scalability:



(a) Weak OpenMP scaling.



(b) Strong OpenMP scaling.

Figure 4: Plots for OpenMP scaling averaged over 5 runs.

For strong scaling, increasing the number of threads from 2 to 24 gives a speedup of approximately 8 rather than the ideal factor of 12. This reduced scalability is likely due in part to the fixed problem size: as more threads are added, the amount of computation assigned to each thread decreases, while thread scheduling and synchronization overheads remain. At higher thread counts, these overheads therefore represent a larger fraction of the total execution time, limiting the achievable speedup.

In contrast, the weak-scaling results show that the workload remains sufficiently large as the number of threads increases. The execution time increases only slightly, from 1.22s to 1.29s, indicating that the additional computational work is effectively compensated for by the additional threads.

4 MPI

4.1 Implementation

MPI requires more careful consideration than the OpenMP implementation because a simple static distribution of the image, for example using `MPI_Scatter` followed by `MPI_Reduce`, does not account for the irregular computational workload of the Mandelbrot set. Different regions of the complex plane can require significantly different numbers of iterations, so assigning a fixed number of pixels to each process can result in substantial load imbalance.

The approach we ended up using is a **master/worker** separation of processes:

- The **master process** is responsible for scheduling the computation and collecting the results from the workers. The image is divided into horizontal sections, referred to as *tiles*, which are dynamically assigned to the workers.
- The **worker processes** receive a tile from the master, perform the Mandelbrot computation for all pixels contained in that tile, and send the computed results back to the master. Once a worker finishes its computation, it is assigned another tile if there is still work available.
- The number of tiles is set to **four times the number of worker processes**. This provides several tiles per worker, allowing the master to distribute the workload dynamically and compensate for the irregular computational cost of different regions of the Mandelbrot set. At the same time, the tiles are kept sufficiently large to avoid excessive communication and scheduling overhead.
- This dynamic distribution helps prevent workers that finish computationally cheaper tiles from remaining idle while other workers are still processing more expensive regions. The choice of four tiles per worker therefore represents a compromise between fine-grained load balancing and the overhead introduced by communicating a large number of small tiles.

This approach therefore trades some communication and scheduling overhead for improved load balancing, which is particularly important for the irregular computational workload of the Mandelbrot set.

4.2 Scaling

Both weak and strong MPI scaling show good scalability as well:

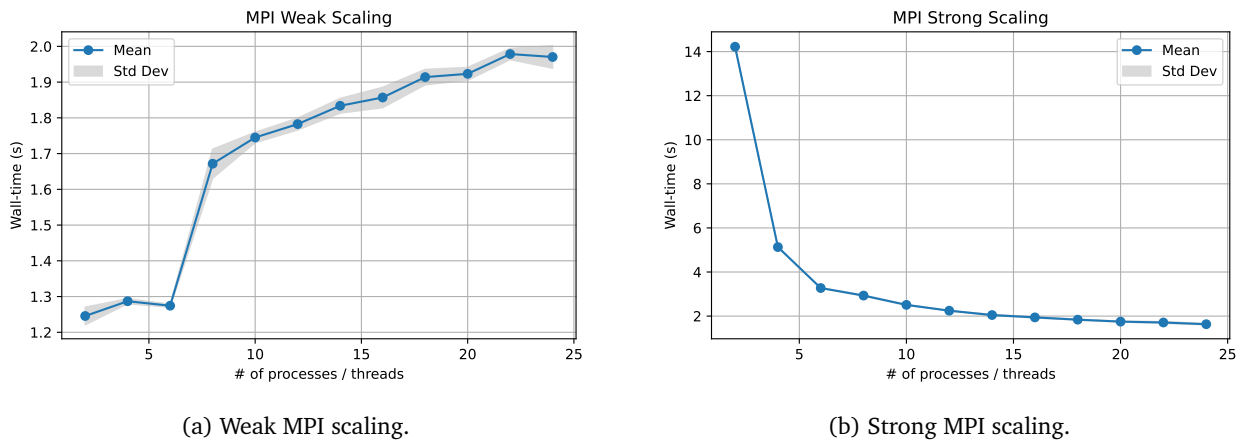


Figure 5: Plots for MPI scaling averaged over 5 runs.

For strong scaling, increasing the number of processes from 2 to 24 reduces the execution time from approximately 14s to 1.8s, corresponding to a speedup of approximately 8 rather than the ideal factor of 12. This is likely due to the communication and scheduling overhead introduced by the master/worker approach. As the number of processes increases, each worker performs less computation while the relative cost of distributing tiles and collecting results increases. Additionally, the processes were distributed across four different cluster nodes, introducing additional inter-node communication overhead.

The weak-scaling results show a larger increase in execution time, from 1.25s with 2 processes to 1.95s with 24 processes. A particularly noticeable spike occurs at 6 processes, which can be attributed to the workload requiring resources from an additional cluster node. This introduces additional inter-node communication and resource-management overhead, resulting in a temporary increase in execution time that is not directly representative of the general scaling trend. Beyond this point, the execution time remains relatively stable, although it gradually increases as the number of processes grows. Ideally, the execution time would remain constant as the problem size increases; the observed increase can be attributed to communication and scheduling overhead, which becomes more significant as the number of processes increases. Nevertheless, the execution time remains relatively stable compared to the increase in the total workload.

5 Conclusions

The results show that both the OpenMP and MPI implementations achieve good scalability for the Mandelbrot set computation. OpenMP provides good scaling on a single shared-memory system, while the MPI implementation is able to efficiently distribute the irregular workload across multiple processes and nodes. In both cases, the scaling is not ideal due to scheduling and communication overhead, which becomes more significant as the number of computational resources increases.

One possible improvement to the MPI implementation would be to allow the master process to perform computation when the number of processes is small. Currently, the master is dedicated to scheduling and communication, leaving one process unavailable for computation. Allowing it to also act as a worker could improve resource utilization, particularly for configurations with a small number of processes.